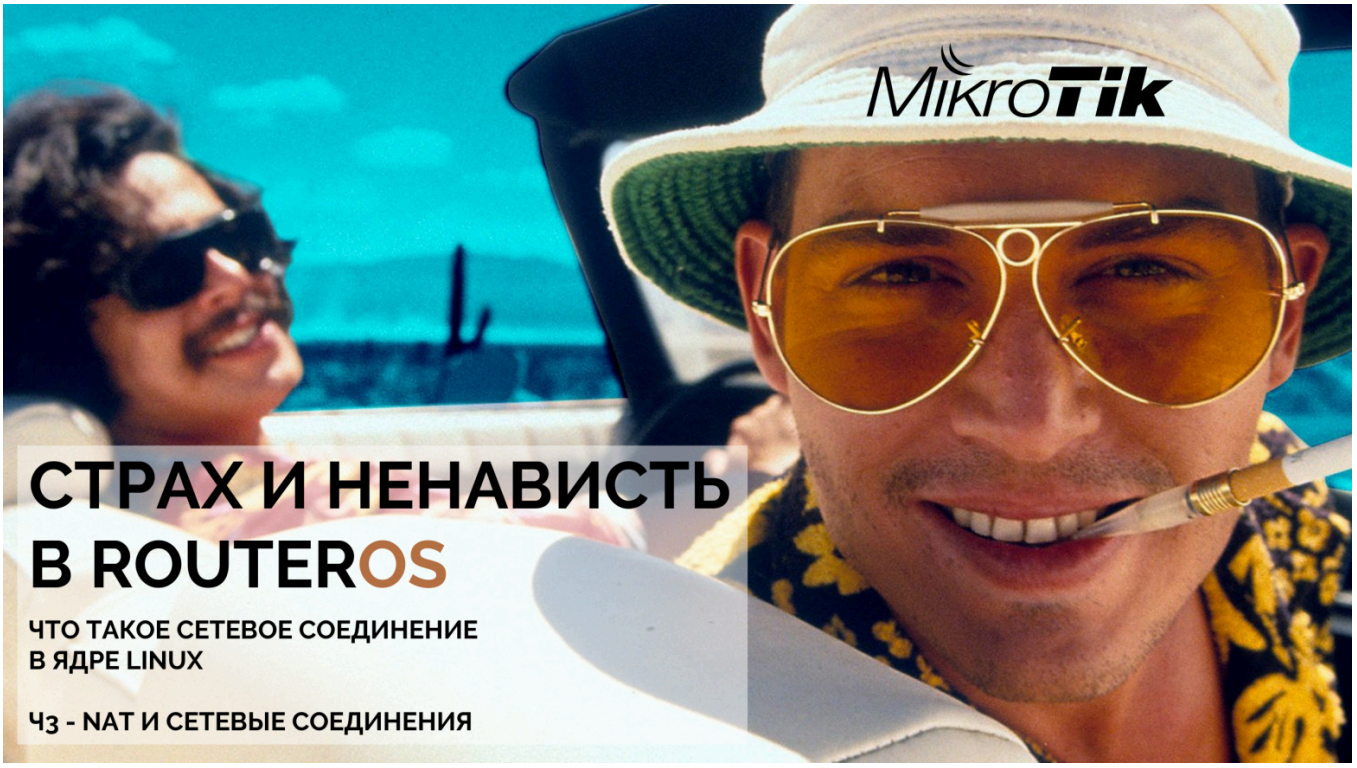
Страх и ненависть в RouterOS: что такое сетевое соединение в ядре Linux (часть 3 — NAT и сетевые соединения)

habr.com/ru/companies/ruvds/articles/590491

olegtsss November 29, 2021



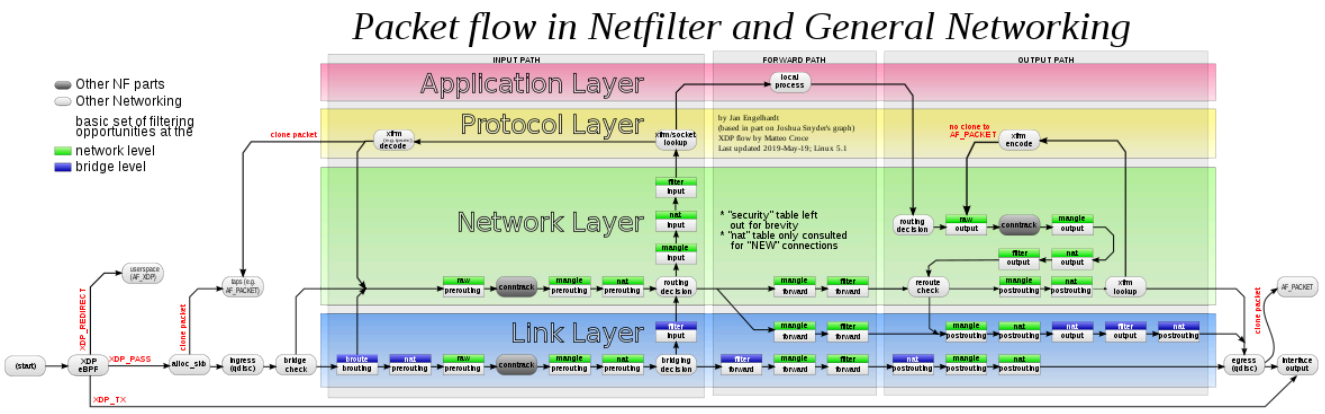
В статье рассмотрено понятие «соединение» для TCP и UDP протоколов в ядре операционной системы Linux на примере работы оборудования MikroTik. Дополнительно рассматриваются особенности работы технологии NAT в указанном контексте. Материалы носят в основном теоретический характер и предназначены для людей, тонко настраивающих Firewall, Qos и маршрутизацию, где им придется непосредственно работать с рассматриваемыми connections.

Мы уже рассмотрели теоретическую части , где я подробно описал сущность сетевого соединения глазами ядра маршрутизатора. Далее была практическая части, где я закрепил информацию, в результате рассмотрения работы прикладного протокола DNS через подсистемы RouterOS.

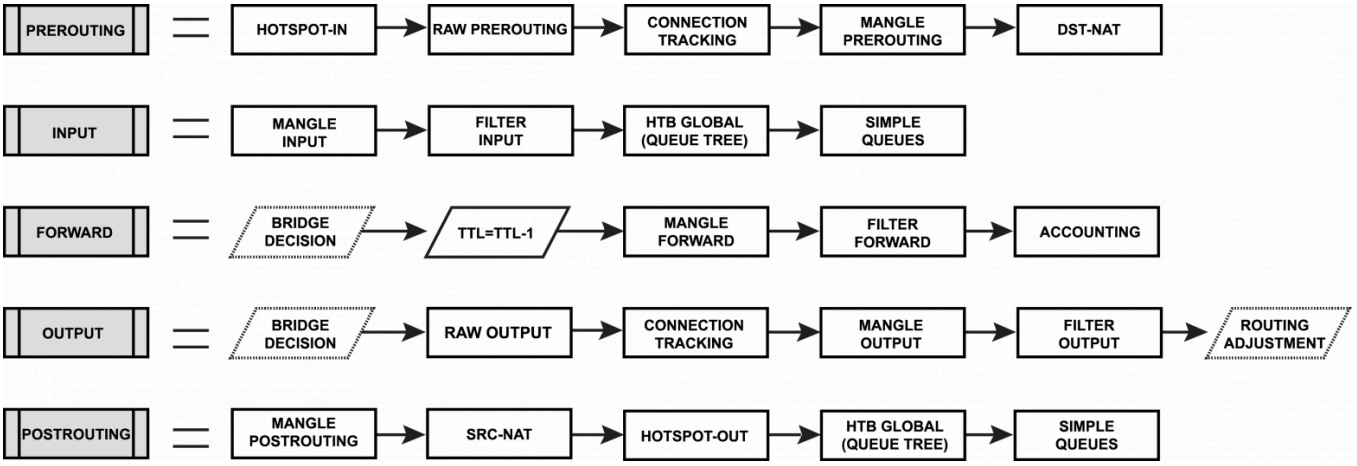
В этой части, цикла статей, речь пойдет о диаграмме потока пакетов, при работе с которой важно понимать сущность рассматриваемого сетевого соединения, а также о не документированной в явном виде особенности работы NAT.

> Цикл статей не предназначен для новичков и может их только запутать. Полагаю, что читатель хорошо знаком с предметом разговора.

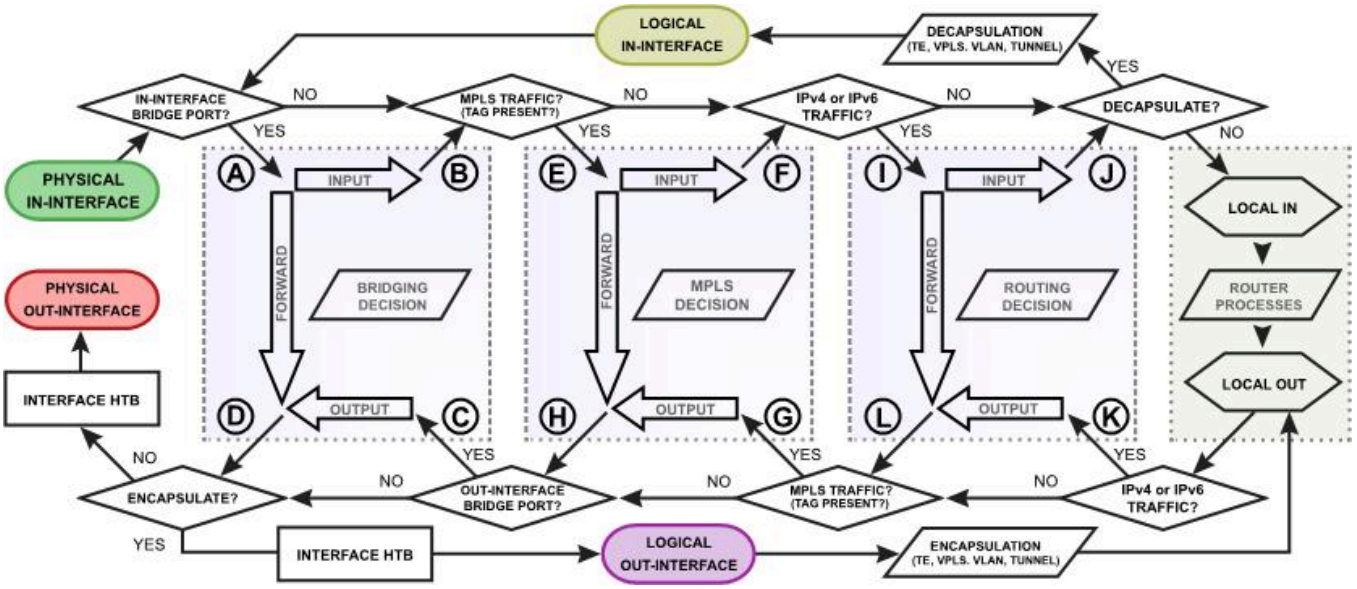
Научиться настраивать MikroTik с нуля или систематизировать уже имеющиеся знания можно на углубленном курсе по администрированию MikroTik. Настало время поговорить об упомянутых в практической части цикла статей цепочках обработки пакетов в ядре Linux. И, конечно же, о NAT. Здесь проясним, почему цепочки выбраны именно такие, как в правилах Mangle, приведенных во второй части цикла статей. Для этого воспользуемся диаграммой потока пакетов, при работе с которой очень важно понимать сущность рассматриваемого сетевого соединения, иначе работа Firewall может стать неожиданной и непонятной. Таких Packetflow Diagram существует много. Вот, например, классическая схема для Linux:



Использовать ее в работу при настройке маршрутизатора очень сложно, наверное, поэтому существуют и другие. Вот одна из таких общих диаграмм непосредственно от компании MikroTik:

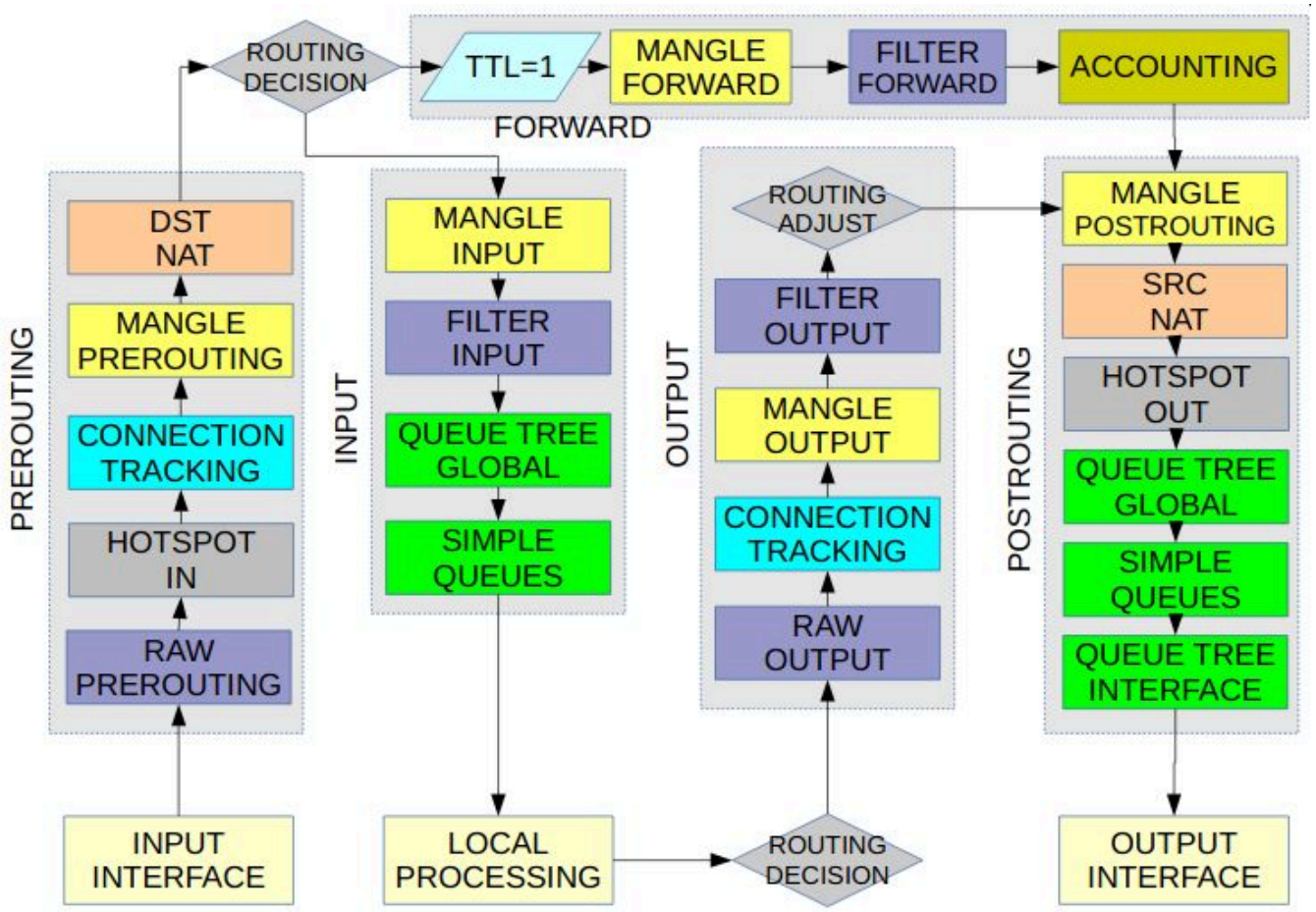


Подробно такие штуки изучаются на курсе «MikroTik Certified Traffic Control Engineer», кому не удобно заниматься очно с сертифицированными тренерами в учебных центрах, тем могу порекомендовать онлайн площадку. На последней схеме уже видны одноименные разделы из Firewall и других частей RouterOS. А вот уже более серьезная официальная разновидность Overall Packetflow Diagram, которой я тоже не пользуюсь, не приросла:

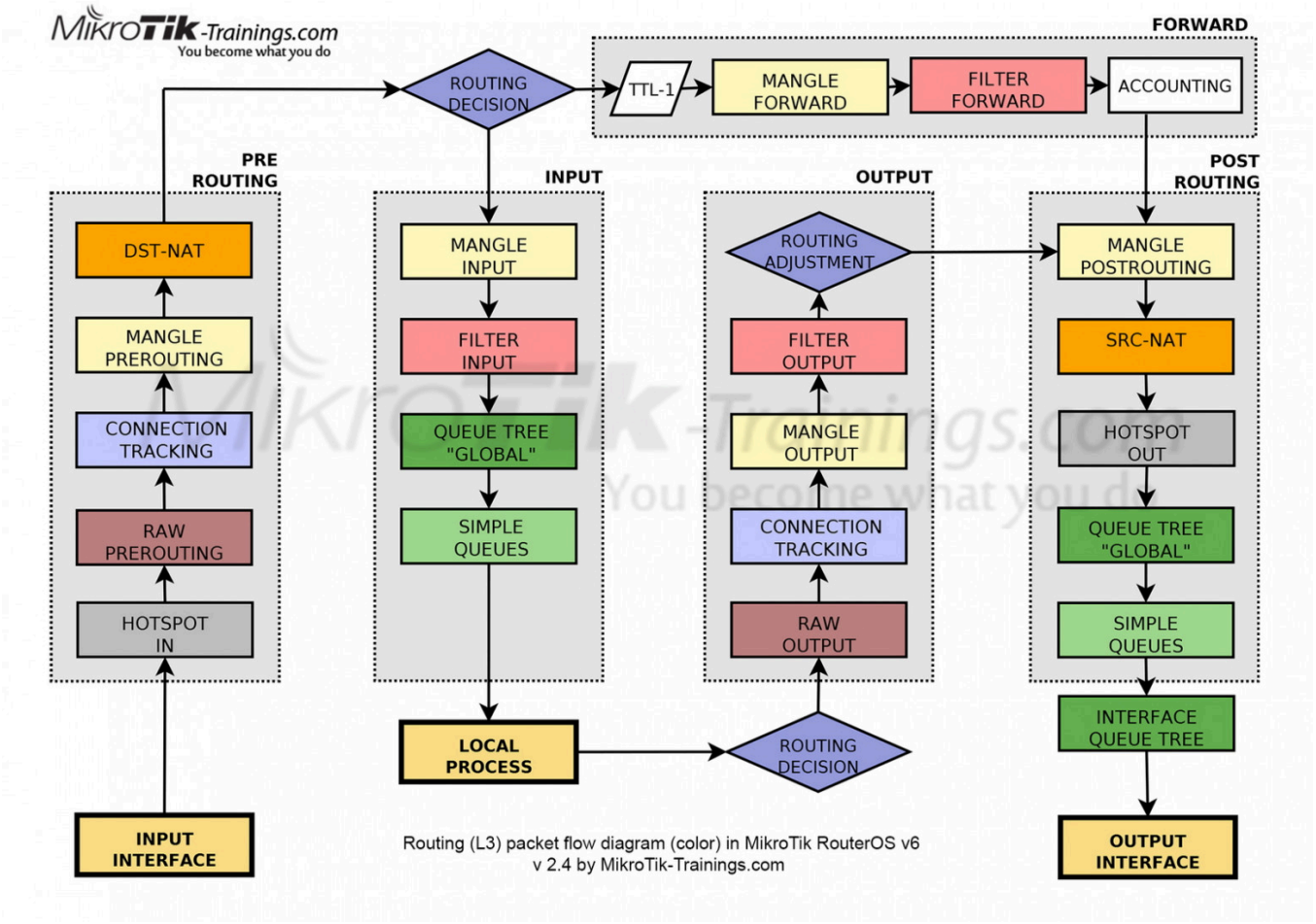


Существуют и не официальные диаграммы, их большое количество, приведу пару примеров. В них повторяются названия блоков, применена немного различная их последовательность:

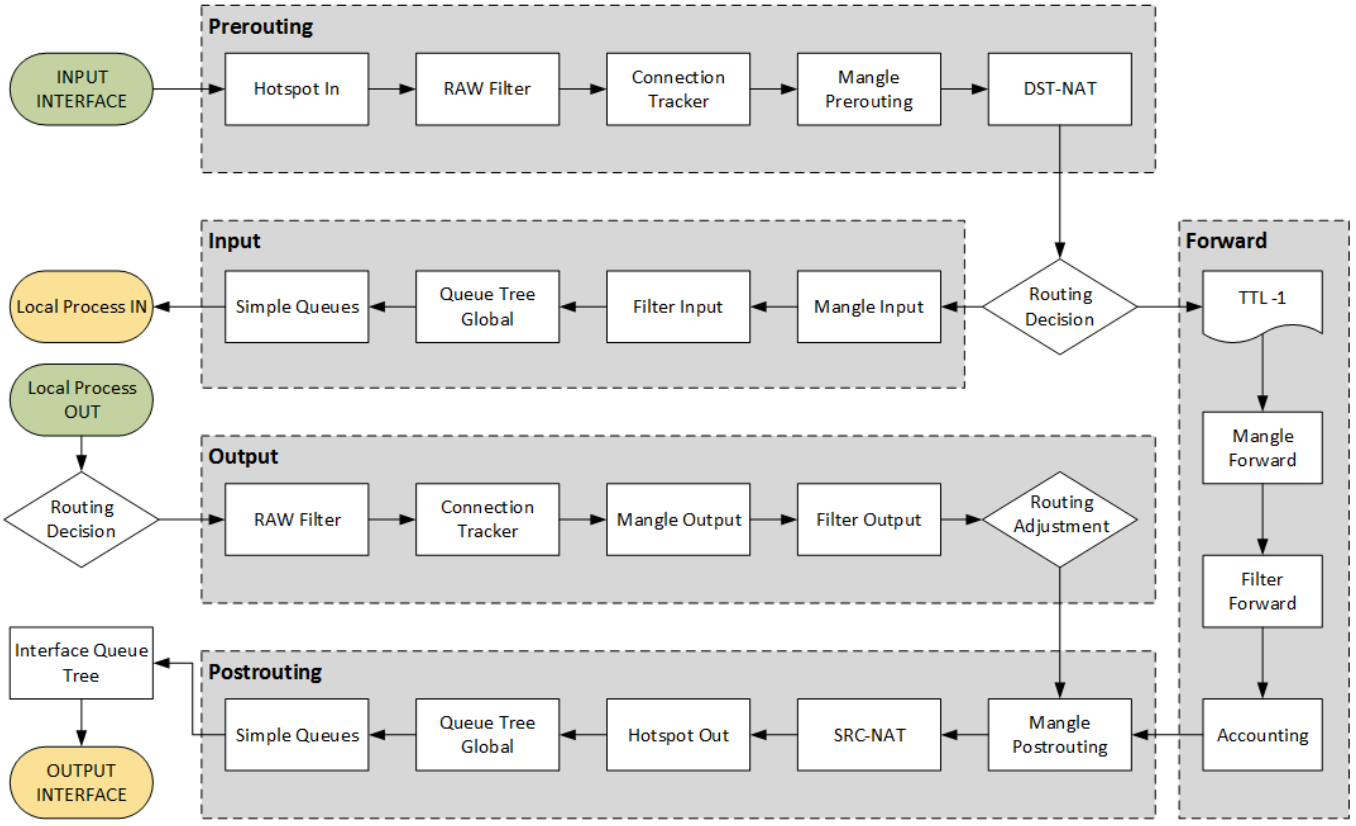
Один вариант:



Другой вариант:



Для себя лично я остановился на такой вот диаграмме и далее по тексту работаю именно с ней:

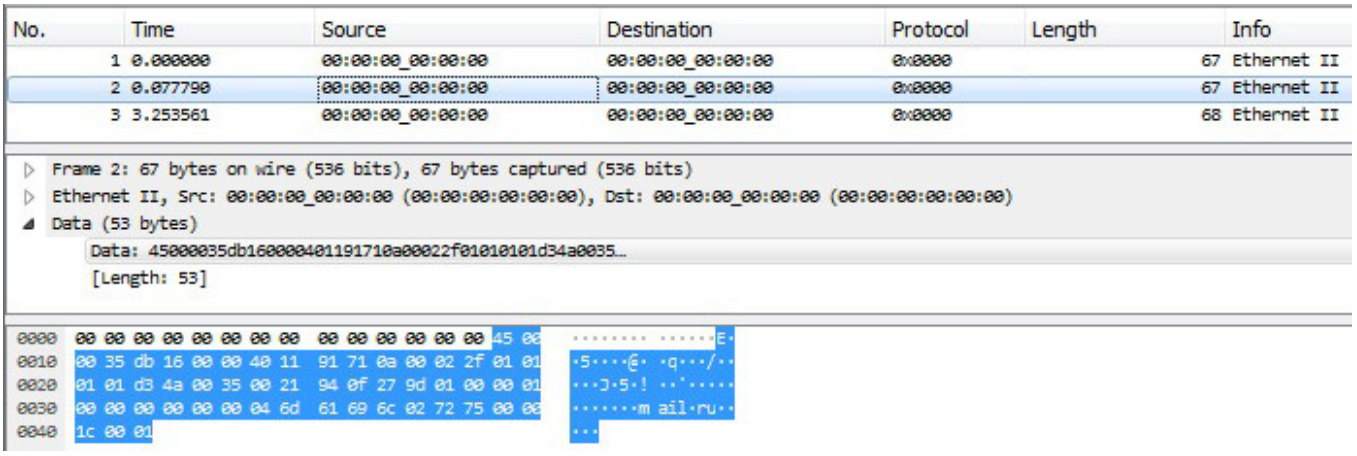


Здесь я нарочно привожу такое количество примерно одинаковых схем прохождения трафика. Это сделано потому, что далее мы увидим ~~скрытую не точность~~ особенность, которая содержится в них всех. Опытные сетевые инженеры, конечно, не найдут в ней ничего нового, но многих она может сбить с толку, а далее наши любимые танцы с бубном. Чтобы понять, о чем идет речь вернется к практическому примеру, рассмотренному во второй части цикла статей. Но для начала сделаем короткое отступление, которое, возможно, кому-то тоже позволит сэкономить время. Ранее мы выполняли зеркалирование трафика именно в цепочке Postrouting. Если это действие выполнить в цепочке Output, то можно получить неожиданный результат:

```
/ip firewall mangle
add action=sniff-tzsp chain=output comment="Sniffer"
packet-mark="DNS catcher MikroTik->DNS Servers Packets" sniff-target=\
    IP_адрес_принимающего_сервера sniff-target-port=37008
```

Объяснить с

Как видно на рисунке, мы имеем чистую полезную нагрузку (DNS query), не обремененную еще канальным, сетевым и транспортным уровнями:



В практической части мы получили соединения и пакеты со следующей маркировкой: «*PC -> MikroTik*», «*MikroTik -> DNS servers*», «*DNS servers -> MikroTik*» и «*MikroTik -> PC*». Теперь выполним трассировку, через какие блоки Packetflow Diagram они пройдут:

- «*PC -> MikroTik*» (input interface → prerouting → input → local process in);
- «*MikroTik -> DNS servers*» (local process out → output → postrouting → out interface);
- «*DNS servers -> MikroTik*» (input interface → prerouting → input → local process in, такая же ситуация как в первом случае);
- «*MikroTik -> PC*» (local process out → output → postrouting → out interface, такая же ситуация как во втором случае).

Теперь, когда мы дошли до Packetflow Diagram, разберем с ее помощью ситуация с некорректной маркировкой из второй статьи представленного цикла. Посмотрим еще раз на иллюстрацию ниже:

Видно, что отображаемая маркеровка перепутана. И перепутана она только в Connection tracking. Дампы каждого типа разработанной нами маркировки доказывают корректность правил Mangle и были представлены ранее. Причина неверных подписей на иллюстрации также подробно разобрана во второй части – перемаркеровка (вернее сказать, домаркеровка). А вот почему *passthrough=no* не спасает ситуацию, поговорим сейчас. DNS запрос от PC поступает на DNS сервер, интегрированный в MikroTik. RouterOS внутри себя создает первое сетевое соединение и навешивает на него марку «*PC->MikroTik Connections*» в блоке Mangle Prerouting.

После ряда действий, описанных во второй части цикла статей, роутер выполняет DNS query response в сторону PC, что являетс►► продолжением и окончанием первого соединения. Вторая половина первого запроса маркируется как «*MikroTik->PC Connections*» в блоке Mangle Output. Повторюсь, это одно и тоже соединение внутри RouterOS, однако половина его вышло с одной меткой, а вторая половина с другой.

Часть соединений несет в себе входящие пакеты от PC, а вторая часть пакетов сгенерирована маршрутизатором. Поэтому параметр *passthrough=no* (примененный кстати к пакетам, а не к соединениям) не скорректировал ситуацию с маркеровкой, так как был применен к различным пакетам, т.е. к пакетам из различных цепочек. Однако половина его вышла с одной меткой, а вторая половина с другой, и Connection tracking отрисовал только ту метку, которая пришла к нему последней, так как Packetflow Diagram имеет два блока Connection tracking:

Для соединений «*MikroTik->DNS Servers Connections*» и «*DNS Servers->MikroTik Connections*» возникает ровно такая же ситуация. Теперь все окончательно должно встать на свои места.

После достаточно подробного рассмотрения Packetflow Diagram перейдем непосредственно к теме статьи, а именно особенностям работы NAT с сетевыми соединениями, созданными внутри операционной системы маршрутизатора. Что такое технология NAT, какие бывают ее разновидности (*source NAT*, *destination NAT*, действия *masquerade*, *port mapping*, *port forwarding*, *carrier-grade NAT* и т.д.), их параметры – все это выходит за предмет повествования, полагаю, что читатель с эти знаком. Детализирую разговор конкретно про соединения с маркой «*MikroTik->PC Connections*». Напоминаю правила Mangle, подготовленные для них:

```
/ip firewall mangle
add action=mark-connection chain=output comment=\
    "DNS catcher MikroTik->PC Connections" new-connection-mark=\
    "DNS catcher MikroTik->PC Connections" passthrough=yes protocol=udp \
    src-address=192.168.1.1 src-port=53
add action=mark-packet chain=output comment=\
    "DNS catcher MikroTik->PC Packets" connection-mark=\
    "DNS catcher MikroTik->PC Connections" new-packet-mark=\
    "DNS catcher MikroTik->PC Packets" passthrough=no
```

Объяснить с

Маркировка была применена в блоке Mangle Output. На всякий случай, попробуем посмотреть эти же пакеты в блоке Raw filter Output:

```
/ip firewall raw
add action=accept chain=output comment="Output packets from Router's DNS server"
protocol=udp src-address=192.168.1.1 src-port=53
```
Объяснить с

Счетчик пакетов отрабатывает. Вроде как все понятно. Попробуем отловить эти же пакеты в блоке SRC-NAT:

```
/ip firewall nat
add action=passthrough chain=srcnat comment="Output packets from Router's DNS server"
disabled=yes packet-mark="DNS catcher MikroTik->PC Packets
```
Объяснить с

А уже здесь счетчик будет иметь нулевое значение. Визуализируем вышесказанное на Packetflow Diagram:

Анализируя приведенную схему можно сделать ошибочный вывод, что под действие NAT, если быть точнее SRC-NAT, должны попасть пакеты с маркировкой «*MikroTik -> PC*» (также как и пакеты с маркировкой «*MikroTik -> DNS servers*»). «*MikroTik -> PC*» пройдут мимо NAT, «*MikroTik -> DNS servers*» попадут под его действие. Здесь в дело вступает такое правило, как «**NAT обрабатывает только один самый первый пакет нового соединения**». И вот в этом аспекте и появляется особенная важность понимания, в чем же сущность сетевого соединения глазами маршрутизатора, подробно рассмотренная в теоретической части цикла статей. На самом деле Packetflow Diagram должна выглядеть немного по-другому (подтверждено компанией MikroTik, запрос «SUP-65046»):

Приведенного уточнения нет ни на одной из представленных выше схем Packetflow Diagram, в связи с чем я и привел их в статье в таком количестве. И явно не упоминается в официальной и часто не официальной документации, что может подкинуть проблем в решении различных инженерных задач при работе с NAT. Это я и считаю особенностью работы NAT с сетевыми соединениями, созданными внутри маршрутизаторов на базе Linux based операционных систем. Повествование выполнено на примере MikroTik, так как технически было проще сделать именно так. Все то же самое работает и непосредственно в Linux.

Заключение

В цикле статей с теоретической и практической точек зрения рассмотрено, что такое сетевое соединение в ядре Linux. Профессионалы, конечно, это отлично знают и понимают. Не корректная трактовка указанного понятия может приводить к неожиданному результату или его отсутствию и стоить часов работы с железками и

документацией к ним. Надеюсь, что кому-то материал будет полезен и позволит сэкономить время, которое лучше потратить на добрые дела.

Часть 1
Часть 2
Часть 3 (вы тут)

RUVDS всем! Если что, RouterOS можно развернуть и на наших виртуальных серверах, как это сделать можно посмотреть здесь.